Kubernetes : mon résumé de révision

Des bases au rolling update et au rollback

Hicham Habchane

19 octobre 2026 — Programme DevOps, Jour 1

1 L'idée de Kubernetes

Docker crée et lance des conteneurs. **Kubernetes** les organise : il les lance, les multiplie, les surveille et les relance.

L'état souhaité : je ne donne pas d'ordre (« lance un conteneur »), je *décris* ce que je veux (« toujours 3 nginx »). Kubernetes corrige la réalité en permanence.

Kubernetes	Restaurant
Conteneur	le plat
Pod	le plateau
ReplicaSet	le chef d'équipe qui compte les plateaux
Deployment	le responsable de salle
template	la fiche recette
labels	l'autocollant sur le plateau
Service	l'accueil du restaurant
Kubernetes / kubectl	le gérant / la télécommande

2 Le cluster (minikube)

Un seul nœud `minikube` : il dirige (control-plane) et fait tourner les Pods. Les conteneurs tournent avec **containerd**.

```
minikube start        # allumer le cluster (après chaque redémarrage du PC)
minikube status       # allumé ou éteint ?
kubectl get nodes     # attendre STATUS = Ready
```

À retenir : erreur `no route to host` ou `connection refused` → le cluster est éteint.

3 Le Pod

La plus petite unité de Kubernetes : une enveloppe autour d'un conteneur, avec un nom et une IP interne.

```
kubectl apply -f pod-nginx.yaml     # créer le Pod
kubectl get pods                    # ContainerCreating puis Running
kubectl describe pod <nom>          # fiche détaillée, Events en bas
kubectl logs <nom>                  # ce que le conteneur affiche
```

Un fichier YAML a toujours 4 parties : `apiVersion`, `kind`, `metadata`, `spec`.

À retenir : un Pod est **jetable**. Un Pod seul supprimé ne revient **jamais**.
`RESTARTS` = redémarrages du *conteneur*, pas du Pod.

4 Le ReplicaSet

Il garde **N Pods identiques** en marche. Il ne regarde **que les étiquettes** (`selector`).

— Pod **en trop** avec la bonne étiquette → il en **supprime** un (le Pod `intrus`).
— Pod orphelin déjà présent avec la bonne étiquette → il l'**adopte**.
— **Limite :** si je change l'image, les Pods existants **ne changent pas**.

```
kubectl get pods -l app=web          # filtrer par étiquette
kubectl describe pod intrus          # Controlled By: ReplicaSet/web-rs = adoption
```

À retenir : 1 sélecteur = 1 seul contrôleur. On n'écrit jamais de ReplicaSet à la main : on écrit un Deployment.

5 Le Deployment

Il crée et pilote les ReplicaSets. Nom d'un Pod : `monsite-86f69f946d-4vcxz` = Deployment – *hash du template* (le ReplicaSet) – identifiant du Pod.

Je change...	**Résultat**
`replicas` (3 → 5)	même ReplicaSet, seul DESIRED change, anciens Pods gardés
le `template` (l'image)	**nouveau** ReplicaSet = nouvelle **révision**

À retenir : 1 révision = 1 ReplicaSet. Les anciens ReplicaSets restent à 0 pour permettre le rollback.

6 Rolling update (mise à jour sans coupure)

```
sed -i 's/image: nginx$/image: nginx:1.27/' deployment-nginx.yaml
kubectl apply -f deployment-nginx.yaml
kubectl rollout status deployment monsite     # suivre en direct
```

Le nouveau ReplicaSet monte (0→5) pendant que l'ancien descend (5→0), petit à petit.

— **maxSurge 25 %** (arrondi au-dessus) : jusqu'à 2 Pods *en plus* (7 au total).
— **maxUnavailable 25 %** (arrondi au-dessous) : toujours au moins 4 Pods disponibles.

À retenir : pendant la mise à jour, 2 versions coexistent : elles doivent rester compatibles.

7 Rollback (retour arrière)

```
kubectl rollout history deployment monsite    # révisions
kubectl rollout undo deployment monsite       # revenir à la version d'avant
```

Le Deployment **réutilise l'ancien ReplicaSet** avec des Pods neufs. C'est un nouveau déploiement : la révision 1 devient la révision 3.

8 Déclaratif, impératif et dérive

— **Déclaratif** (bonne pratique) : je modifie le fichier puis `apply`. Le fichier dans Git fait foi.
— **Impératif** : `kubectl scale`, `rollout undo`. Le fichier n'est pas modifié.
— Cluster ≠ fichier = **dérive** (drift) : le prochain `apply` remet l'ancien état.

À retenir : après un rollback, je remets le fichier d'accord avec le cluster. `apply` répond `created`, `configured` ou `unchanged`.

9 Services (à venir)

Les Pods changent d'IP. Le Service donne **un nom et une adresse fixe** devant un groupe de Pods (par `selector`) et répartit les requêtes. **ClusterIP** : interne au cluster. **NodePort** : accès externe (ports 30000–32767). **port-forward** : tunnel temporaire vers mon PC.

10 Questions d'entretien

Question	Réponse courte
Pod, ReplicaSet, Deployment ?	Pod = exécute ; ReplicaSet = maintient N Pods par étiquettes ; Deployment = pilote les ReplicaSets, rolling update et rollback.
Comment K8s garde 3 replicas ?	Le ReplicaSet compare sans arrêt le nombre de Pods étiquetés au nombre voulu, puis crée ou supprime.
Pourquoi garder les vieux ReplicaSets ?	Pour le rollback.
maxSurge / maxUnavailable ?	Pods en plus autorisés / Pods indisponibles au maximum → pas de coupure.
Déclaratif ou impératif ?	Déclaratif : le fichier dans Git fait foi, sinon dérive.